Lecture 9 -- Fine-Tuning and LoRA

Full fine-tuning updates every weight in a pretrained model. For a 7B parameter model that means 7 billion gradients and 7 billion optimizer states in memory -- Adam alone doubles that -- which is why full fine-tuning of large models is usually out of reach on a single GPU.

LoRA (Low-Rank Adaptation) freezes the original weight matrix W entirely and learns a much smaller update instead. Rather than learning a full delta-W the same shape as W, LoRA factors the update into two small matrices A and B, of rank r, so that delta-W = B times A. If W is d by k, A is r by k and B is d by r, with r chosen far smaller than d or k -- often 8, 16, or 32. The number of trainable parameters drops from d times k to r times (d plus k), which for typical transformer layer sizes is a reduction of two to three orders of magnitude.

At inference time the adapter can be merged back into the base weights (W plus B times A) with zero added latency, or kept separate so a single base model can swap between many task-specific adapters cheaply -- this is the mechanism that makes serving many fine-tuned variants of one base model practical.

Quantization is a complementary technique: it reduces the precision each weight is stored at, typically from 16-bit floats to 8-bit or 4-bit integers, cutting memory further. QLoRA combines both: the frozen base model is quantized to 4 bits, while the small LoRA adapters are trained in higher precision on top of it -- which is what makes it possible to fine-tune a 65B parameter model on a single consumer GPU.

The key tradeoff across all of this is rank r. Too low, and the adapter lacks the capacity to represent the target task’s shift in behaviour. Too high, and LoRA approaches the cost of full fine-tuning without a clear accuracy benefit -- in practice r is usually chosen well below the point where returns diminish, since low-rank updates generalize better than high-rank ones on small fine-tuning datasets.